Exercices 1.2 and 1.3: Perceptron Learning Algorithm

Francisco Gutiérrez

Exercise 1 Exercise 1.2 — Perceptron for Spam Detection

(a) Keywords likely to get a large positive weight

Words that appear disproportionately often in spam and rarely in legitimate mail will pick up large positive weights, since their presence pushes the score toward $+1$. Typical examples: *"free," "win," "winner," "prize," "click here," "viagra," "guarantee," "credit," "cash," "offer," "nigerean,"," "king,"," "urgent," "buy now."*

(b) Keywords likely to get a negative weight

Words that occur often in legitimate mail but rarely in spam will get negative weights, since their presence pushes the score toward -1. Examples: names of frequent contacts, *"meeting," "attached," "project," "invoice," "regards," "conference,"* or technical/work-specific jargon particular to the user's normal correspondence.

(c) Parameter that controls how many borderline messages are classified as spam

The **threshold/bias term** (often written w_0 or b) directly controls this. The perceptron's decision is

$$h(\mathbf{x}) = \text{sign}\left(\mathbf{w}^T\mathbf{x} + b\right).$$

Changing b shifts the decision boundary without changing its orientation, so it directly determines how many borderline (near-boundary) messages fall on the spam side versus the legitimate side — i.e., it trades off false positives against false negatives.

Exercise 2 Exercise 1.3 — The PLA Update Moves in the Right Direction

Recall the update rule: if $\mathbf{x}(t)$ is misclassified,

$$\mathbf{w}(t+1) = \mathbf{w}(t) + y(t)\mathbf{x}(t).$$

(a) Show $y(t)\mathbf{w}^T(t)\mathbf{x}(t) < 0$

Since $\mathbf{x}(t)$ is misclassified by $\mathbf{w}(t)$, the predicted sign disagrees with the true label:

$$\text{sign}\left(\mathbf{w}^T(t)\mathbf{x}(t)\right) \neq y(t).$$

This means $\mathbf{w}^T(t)\mathbf{x}(t)$ and $y(t)$ have opposite signs. Since $y(t) = \pm 1$, multiplying a quantity by something of opposite sign flips it negative:

$$y(t)\,\mathbf{w}^T(t)\mathbf{x}(t) < 0.$$

(b) Show $y(t)\mathbf{w}^T(t+1)\mathbf{x}(t) > y(t)\mathbf{w}^T(t)\mathbf{x}(t)$

Using the update rule:

$$\mathbf{w}^T(t+1)\mathbf{x}(t) = (\mathbf{w}(t) + y(t)\mathbf{x}(t))^T\,\mathbf{x}(t) = \mathbf{w}^T(t)\mathbf{x}(t) + y(t)\,\mathbf{x}^T(t)\mathbf{x}(t).$$

Multiply both sides by $y(t)$, and use $y(t)^2 = 1$:

$$y(t)\mathbf{w}^T(t+1)\mathbf{x}(t) = y(t)\mathbf{w}^T(t)\mathbf{x}(t) + \|\mathbf{x}(t)\|^2.$$

Since $\|\mathbf{x}(t)\|^2 \geq 0$ (strictly positive as long as $\mathbf{x}(t) \neq \mathbf{0}$):

$$y(t)\mathbf{w}^T(t+1)\mathbf{x}(t) > y(t)\mathbf{w}^T(t)\mathbf{x}(t).$$

(c) Why this is a move "in the right direction"

The quantity $y(t)\mathbf{w}^T(t)\mathbf{x}(t)$ is a **signed classification score**: it is negative when $\mathbf{x}(t)$ is misclassified and would be positive if it were classified correctly. Part (a) shows this quantity starts negative (misclassified), and part (b) shows the update strictly *increases* it. So even though one update may not be enough to flip $\mathbf{x}(t)$ to the correct side, it always pushes the score for that specific point closer to (and eventually across) zero — i.e., closer to correct classification. That is the precise sense in which $\mathbf{w}(t+1)$ is "more correct" for $\mathbf{x}(t)$ than $\mathbf{w}(t)$ was.

Additional Questions

a) Does the algorithm always correctly separate the data?

Only if the data is **linearly separable**. If there exists some $\mathbf{w}^*$ that classifies all points correctly, then PLA is guaranteed to find *a* separating hyperplane (not necessarily $\mathbf{w}^*$ itself, just *some* separator). If the data is **not** linearly separable, PLA will never converge — it will keep correcting one point at the expense of misclassifying another, cycling forever, with no guarantee it settles anywhere.

b) Does it stop in a finite number of steps? How is convergence guaranteed?

Yes — **if the data is linearly separable**, the **Perceptron Convergence Theorem** guarantees PLA halts after a finite number of updates. Sketch of the argument:

- Let $\mathbf{w}^*$ be a separating vector with margin $\rho = \min_n y_n \mathbf{w}^{*T}\mathbf{x}_n > 0$, and let $R = \max_n \|\mathbf{x}_n\|$.
- Each update increases the alignment $\mathbf{w}(t)^T\mathbf{w}^*$ by at least ρ, so it grows **at least linearly** in t.
- Each update increases $\|\mathbf{w}(t)\|^2$ by at most R^2, so $\|\mathbf{w}(t)\|$ grows **at most as** $\sqrt{t}$.
- Since $\cos(\text{angle}) = \dfrac{\mathbf{w}(t)^T\mathbf{w}^*}{\|\mathbf{w}(t)\|\|\mathbf{w}^*\|} \leq 1$, and the numerator grows linearly while the denominator grows like $\sqrt{t}$, this forces a bound:
$$t \leq \frac{R^2\|\mathbf{w}^*\|^2}{\rho^2}.$$
- So the number of updates (mistakes) is finite and bounded — the algorithm **must** stop.

c) Why does it work?

Intuitively: every time PLA makes an update, it uses a currently misclassified point to nudge the weight vector so that this particular point's score improves (part (b) above). At the same time, the convergence proof shows this nudging cannot go on forever without the weight vector becoming increasingly well-aligned with *some* valid separator $\mathbf{w}^*$ — the alignment (measured via the dot product) grows steadily while the vector's magnitude cannot grow fast enough to "cheat" that alignment indefinitely. So each local, greedy correction (fix whichever point is currently wrong) accumulates into global progress toward a hyperplane that gets everything right, precisely because a separating hyperplane exists to converge toward. Without separability, there is no such $\mathbf{w}^*$ to converge to, and the guarantee breaks down.